WUNDERDOG

Admicom x Wunderdog – Evals & Cloud Agents

DevDays 20.–21.10.2026 @ Jyväskylä

Admicom x Wunderdog – Evals & Cloud Agents

Opening slide. Say who you are and what the next hour is about: how to know an AI agent actually does its job (evals), and how to let agents work on their own in the cloud (cloud agents). Tomorrow's workshop builds on this, so everything today is something we will use.

Key points
- Two topics: evaluations and cloud agents
- Talk first, hands-on workshop after
- Goal: by Wednesday afternoon each group has a working loop

Workshop facilitators



Juuso Korhonen

Technology Lead

- 10+ years of full stack web, mobile and cloud experience
- 15+ years of programming
- Technical Lead specializing in rapid idea-to-production delivery within weeks
- Claude as the daily driver, Pi for funsies



Mikko Vallaskivi

Technology Lead

- Software development generalist
- 10+ years of full stack web, mobile and cloud experience
- 15+ years of programming
- Copilot/Codex/Claude Code

Workshop facilitators

Introduce the two of you. Juuso first, then Mikko. Keep it short: who you are, what you do at Wunderdog, and why you are the ones running this workshop. Say that both of you will be around during the workshop to help the groups.

Key points

- Juuso Korhonen, Technology Lead: 10+ years of full stack web, mobile and cloud, 15+ years of programming, fast idea-to-production delivery, uses Claude daily and Pi for fun
- Mikko Vallaskivi, Technology Lead: software development generalist, 10+ years of full stack web, mobile and cloud, uses Copilot, Codex and Claude Code daily
- Both of you help the groups during the workshop

Source

Admicom x Wunderdog workshop deck (Google Slides), slide 5

Two themes

01

Evaluations

Know when it works

Trust → autonomy

02

Cloud agents

Let it work while you don't

Two themes

The whole talk is two ideas. First, evals tell you if an agent works, with numbers instead of gut feeling. Second, cloud agents let the agent keep working when you are not at your laptop. The arrow at the bottom is the link: you only give an agent more freedom once evals show you can trust it.

Key points

- Evals = know when it works
- Cloud agents = let it work while you don't
- Trust comes first, then more freedom for the agent

WUNDERDOG

01

Evaluations

01 Evaluations

Start of part one. Everything in this part comes from Anthropic's engineering post on agent evals and the Claude Code plugin eval docs.

Key points

- Part 1: how to test an agent properly

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
https://code.claude.com/docs/en/plugin-evals

Vibes don't scale

PROMPT Find the lessons in last week's review comments



Same prompt. Different outcome.

Vibes don't scale

Same prompt, three runs, and one of them goes wrong. Agents are not deterministic: the same input can give a different result each time. If you try it once and it looks fine, you have not proven anything. That is why evals run the same task several times.

Key points

- One good run proves nothing
- Same prompt can give different results
- Run each task several times and count passes
- The example prompt is our workshop skill: review-lessons

Anthropic's post: run multiple trials because outputs vary between runs.

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Anatomy of an eval



Task

Input and success criteria



Trials

Several attempts per task

Transcript

Every step the agent took

Outcome

Final state of the world

tests pass

skill used

rubric

Graders

Each checks one thing

67%

Score

Pass rate across trials

Anatomy of an eval

The words we will use for the rest of the talk, taken from Anthropic's eval post. A task is one test with an input and success criteria. A trial is one attempt at that task; you run several. The transcript is the full record of a trial: outputs, tool calls, reasoning. The outcome is the real end state, for example whether the file or database row actually exists. Graders each check one thing, and the score is the pass rate across trials. In the workshop phases we call a trial a run.

Key points

- Task: input + what counts as success
- Trial: one attempt, run several
- Transcript: everything the agent did
- Outcome: the actual end state, not what the agent said
- Grader: checks one thing; score = pass rate (67% = 2 of 3)

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Grade the outcome, not the claim

WHAT THE AGENT SAYS

Fixed the flaky test. CI is green.

CLAIM ≠ OUTCOME

WHAT THE REPO SAYS

```
$ run-tests auth.spec --repeat 10
✓ 7 runs passed
× 3 runs failed   token expiry
```

- Check the end state
- Not the agent's report

Grade the outcome, not the claim

The agent says the flaky test is fixed. Running the test 10 times shows 3 failures. Agents report success confidently even when it is not true, so graders should check the real result in the repo, not the agent's message. Anthropic's post uses a flight booking example: the agent says 'booked', but the check is whether the booking exists in the database.

Key points

- Agents can say 'done' when it isn't
- Check the end state: tests, files, data
- Flaky tests need repeated runs to catch
- The test output here is example data

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Three kinds of graders

Code

Unit tests · regex · tool-call checks

+ Fast, cheap, repeatable
– Brittle on nuance

Model

Rubrics · pairwise · judge panels

+ Handles open-ended output
– Needs calibration

Human

Expert review · spot checks

+ Gold standard
– Slow and expensive

Prefer deterministic

Escalate only when needed

Three kinds of graders

Three ways to grade, all from Anthropic's post. Code graders (tests, regex, tool-call checks) are fast, cheap and give the same answer every time, but break when a valid answer looks slightly different. Model graders (an LLM with a rubric) handle open-ended output but are not deterministic and need checking against human judgement. Human review is the gold standard but slow and expensive. Anthropic's advice: use deterministic graders where you can.

Examples

- Code grader: requires `pnpm test`, so it fails an agent that ran `pnpm vitest run src/cart.test.ts`
- Model grader: accepts both commands, but can fail the same transcript on a second run
- Human review: sees that the agent edited the tests to make them pass

Key points

- Code: fast, cheap, repeatable, can be too strict
- Model: flexible, needs calibration with humans
- Human: best quality, slow and expensive
- Default to code graders, add others only when needed

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

pass@k vs pass^k

Run the same task k times. Does any run succeed, or does every run?

pass@k

$1 - (1 - p)^k$

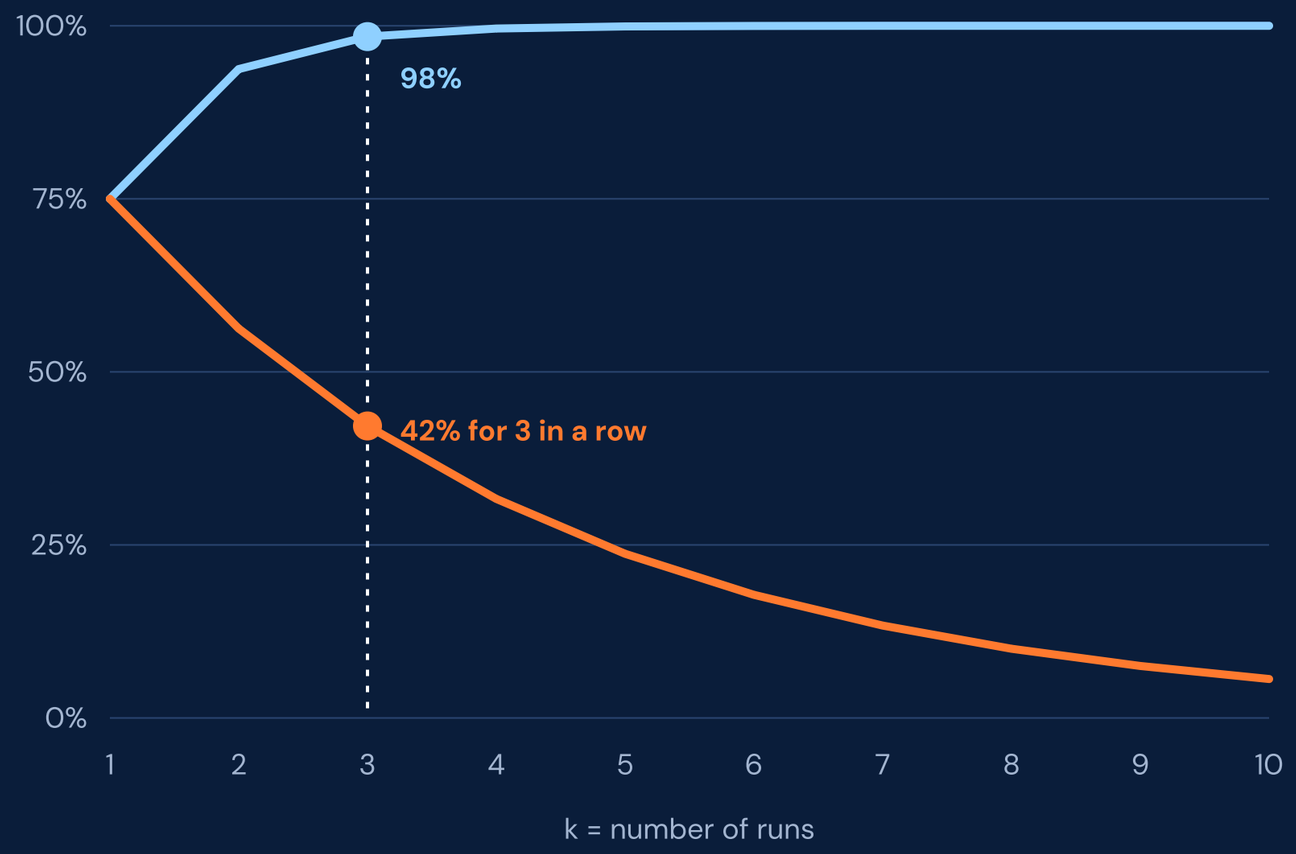
At least one run succeeds

pass^k

p^k

Every run succeeds

p 75%



pass@k vs pass^k

Two ways to read the same runs. pass@k is the chance that at least one of k tries works. pass^k is the chance that all k tries work. With a 75% success rate per try, pass@3 is about 98%, but pass^3 is only about 42% (this exact example is in Anthropic's post). Pick the one that matches the job: if a human picks the best of several tries, pass@k is fine. If the agent runs alone every night, you need pass^k.

Examples

- pass@k: three cloud agents try the same bug fix and you merge the best PR, so one success is enough (98% at 75% each)
- pass^k: a nightly agent labels new issues alone, so at 75% per night a clean work week is only 24%

Key points

- pass@k = at least one success: 1 − (1 − p)^k
- pass^k = every run succeeds: p^k
- 75% per run → pass@3 ≈ 98%, pass^3 ≈ 42%
- Unattended agents (routines) need pass^k

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Capability vs regression

Capability

What can it do yet?



64%

Regression

Does it still work?

three-parts

GRADUATES WHEN IT PASSES RELIABLY



98%

Capability vs regression

Two kinds of eval suites. Capability evals ask what the agent can do well and should start with a low pass rate, so there is room to improve. Regression evals check that things that already work keep working, and should pass close to 100%. When a capability case passes reliably, it moves into the regression suite (Anthropic calls this graduating). The capability bar starts at 18% and climbs to 64% as the agent improves; both numbers and the 98% are example values.

Examples

- Capability: "add discount codes to the cart, with tests" passes 2 of 5 runs, which shows room to improve
- Regression: "run the linter before finishing" passes every run; a CLAUDE.md edit that drops it to 3 of 5 is caught
- Graduating: once the discount task passes 5 of 5 for several days, it moves to the regression suite

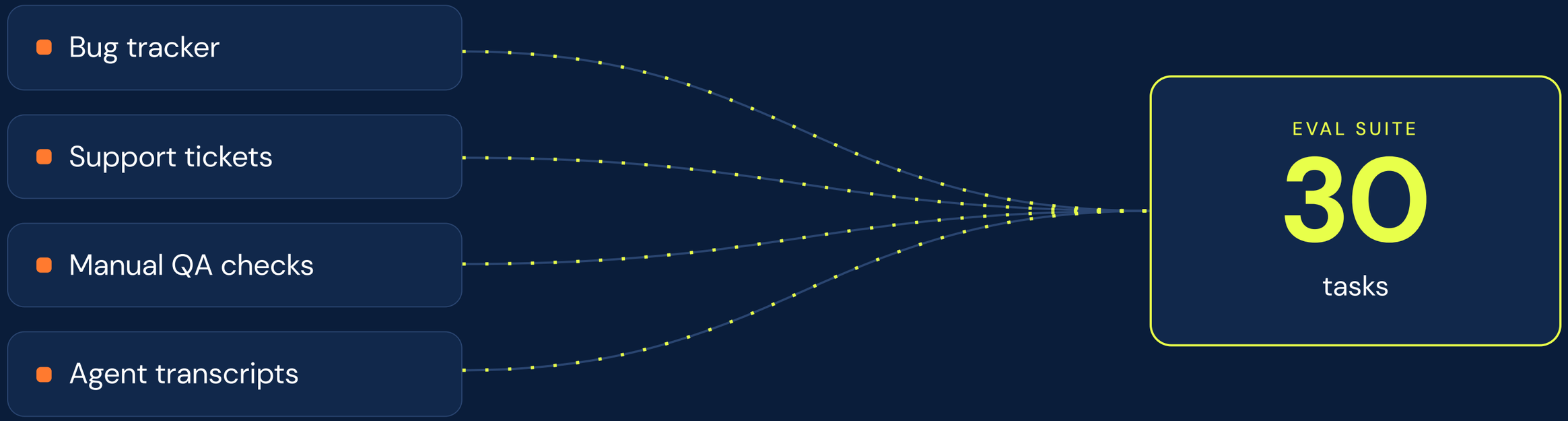
Key points

- Capability: hard cases, low score is expected
- Regression: should be near 100%
- A case moves to regression once it passes reliably
- three-parts is an example case for our review-lessons skill

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Start from real failures



20–50 tasks is enough to start

Start from real failures

Don't invent test cases from scratch. Take them from things that actually went wrong: the bug tracker, support tickets, the manual checks you already do, and agent transcripts where it failed. Anthropic's post says 20 to 50 simple tasks from real failures is a great start. You don't need hundreds.

Key points

- Use real failures, not made-up cases
- Sources: bug tracker, support queue, manual checks, transcripts
- 20–50 tasks is enough to start

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Anatomy of a plugin eval

```
evals/
└─ three-parts/
   ├─ prompt.md
   └─ graders/
      ├─ where-line.md
      └─ skill-fired.md
```

prompt.md

```
---
max_turns: 20
allowed_tools: [Read, Skill]
---
Find the lessons in the review comments in comments.json.
Write the report to report.md.
```

where-line.md

```
type: regex
target:
  source: file
  path: report.md
pattern:
  'Where a change could help:'
```

- Grades the result

skill-fired.md

```
type: tool_used
tool: Skill
input_match:
  'review-lessons'
```

- Grades the steps

Anatomy of a plugin eval

This is how Claude Code's plugin evals look on disk. Each case is a folder under evals/ with a prompt.md and a graders folder. The prompt.md frontmatter sets limits like max_turns and allowed_tools. allowed_tools only grants read-only tools (Read, Glob, Grep, Skill and similar); writing files needs --allow-tools Write when you run it, as on the next slide. Each run starts in an empty folder, so input files like comments.json are added with case.yaml (add_dirs or a scaffold script). Here we use two graders: a regex on report.md checks the result, and tool_used checks that the review-lessons skill was actually called.

Key points

- One folder per case: prompt.md + graders/
- max_turns default is 10, max 200
- allowed_tools grants read-only tools; others need --allow-tools
- One grader checks the result, one checks the steps the agent took to get there

Sources

https://code.claude.com/docs/en/plugin-evals

Six grader types

FREE · FROM TRANSCRIPT AND FILES

`regex`	Pattern in reply, trace or file
`tool_used`	Tool called, n times
`tool_order`	A before B
`file_exists`	File was created

JUDGE MODEL · COSTS A CALL

`llm`	Rubric, 2 of 3 votes
`baseline`	At least as good as a reference transcript

One grader on the result.
One on the steps the agent took to get there.

Six grader types

The six grader types in claude plugin eval. Four are free because they only read the transcript and files: regex (a pattern in the reply or a file), tool_used (a tool was called, with min/max counts), tool_order (tool A called before tool B), file_exists (a file was created). Two call a judge model and add cost: llm (passes on a rubric in at least 2 of 3 votes) and baseline (at least as good as a saved reference transcript).

Key points

- Free: regex, tool_used, tool_order, file_exists
- Cost a model call: llm, baseline
- llm needs 2 of 3 votes to pass
- Good habit: one grader on the result, one on the steps the agent took to get there

Sources

https://code.claude.com/docs/en/plugin-evals

Run it

```
$ claude plugin eval init
drafts cases and graders, tries them once

$ claude plugin eval . --allow-tools Write

CASE                    WITH    W/OUT   Δ
three-parts             1.00    0.33    +0.67
names-one-file          0.67    0.00    +0.67
links-each-comment      1.00    0.67    +0.33
skips-one-off-nit       1.00    1.00    0.00

report → evals/results/<timestamp>/report.html
```

- 3 runs per case
- Baseline runs without the plugin
- Gate CI on the threshold

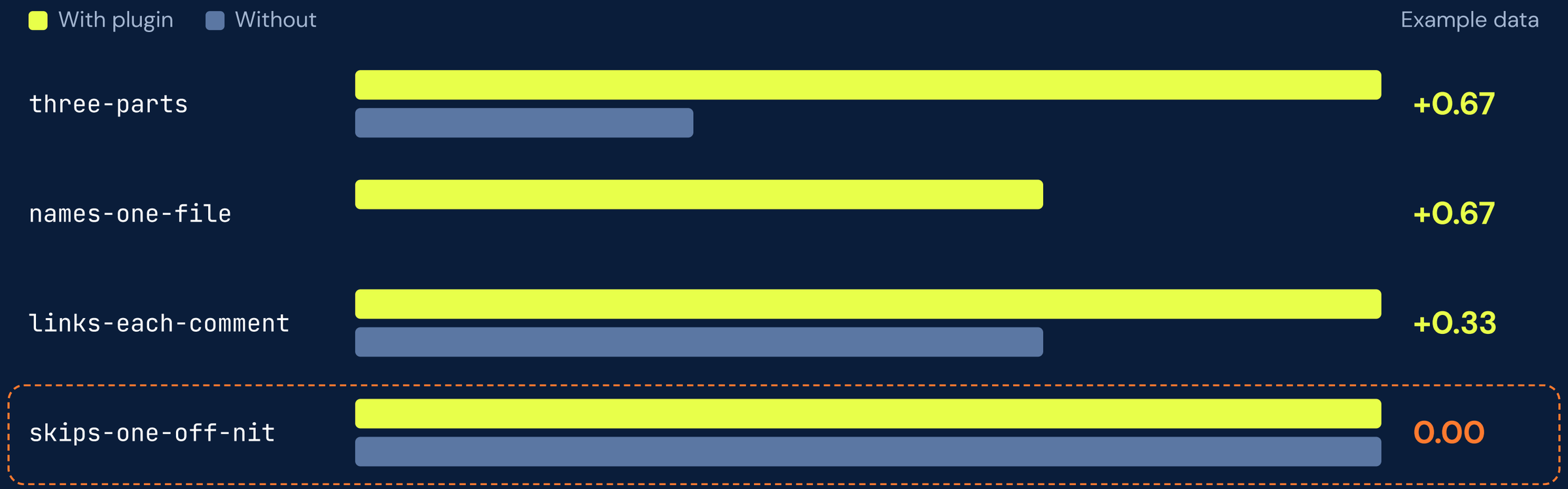
Run it

Two commands. 'claude plugin eval init' asks about your plugin, drafts cases and graders, tries them once and writes the files. 'claude plugin eval .' runs the suite. Each case runs 3 times by default, with the plugin and without it, so you get a WITH score, a W/OUT score and the difference. Results go to evals/results/<timestamp>/ with a report.html. In CI, --threshold makes the command fail if a case scores below the limit (default 1.0). The numbers in the table are example data.

Key points
• init drafts the suite for you
• 3 runs per case by default
• Runs with and without the plugin and shows the difference
• --threshold fails CI below a score
• --allow-tools Write because this case writes a file

Sources
https://code.claude.com/docs/en/plugin-evals

Read the delta



Passes without the plugin → the plugin isn't why it passes

Read the delta

The difference column is the important one. It shows what the plugin adds compared with plain Claude. If a case passes just as well without the plugin (skips-one-off-nit, 0.00), the plugin is not the reason it passes. Either drop the case or make it harder. Big positive numbers show where the skill really helps.

Key points

- Δ = with plugin minus without
- 0.00 means the plugin adds nothing on that case
- Example data, not real results

Sources

https://code.claude.com/docs/en/plugin-evals

Test both directions

SHOULD TRIGGER

- What do reviewers keep flagging in our PRs?
- Group last week's review comments by theme
- Which review comments should become rules?

SHOULD NOT TRIGGER · NEAR MISSES

- Review this pull request
- Reply to the comments on PR #412
- Write release notes from last week's PRs

~20 queries | 60 / 40 train · test | 3 runs per query | Pick the description by held-out score

Test both directions

A skill only helps if it loads at the right time. Claude decides that from the skill's description, so test the description too. Write prompts that should trigger the skill and near misses that should not, like 'review this PR', which sounds close but is a different job. The Agent Skills guide suggests about 20 queries, a 60/40 split between train and held-out sets, 3 runs per query, and picking the description with the best held-out score. The skill-creator plugin automates this kind of loop.

Key points

- Test when the skill should load and when it shouldn't
- Near misses are the most useful negatives
- ~20 queries, 60/40 split, 3 runs each (Agent Skills guide)
- Choose the description by held-out score, not train score

Sources

https://agentskills.io/skill-creation/optimizing-descriptions
https://code.claude.com/docs/en/skills

Pitfalls

Ambiguous task

Two experts would agree on pass or fail

Rigid grader

`"96.12"` rejected for `96.124991…`

Shared state

Leftover files inflate scores between runs

Unread transcripts

Read them. Every failure, every time.

42% → 95% Opus 4.5 on CORE-Bench. Same model: only the eval was fixed (grader, task specs, setup).

Pitfalls

Four common mistakes from Anthropic's post. Ambiguous task: if two domain experts would not agree on pass or fail, the task is unclear. This is a thought test: imagine giving them the task and the result. Rigid grader: CORE-Bench marked '96.12' wrong because it expected '96.124991...'. Shared state: leftover files or cache from earlier runs can make results look better or fail together; the post mentions Claude reading git history from previous trials. Unread transcripts: always read them, they show why something failed.

The line at the bottom shows what these pitfalls can cost. Opus 4.5 first scored 42% on CORE-Bench. An Anthropic researcher found rigid grading, ambiguous task specs and tasks that could not be reproduced exactly. After fixing those bugs and using a less constrained setup, the same model scored 95%. The model did not change; the eval did.

Key points

- Two domain experts would give the same verdict
- Too-strict graders give fake failures
- Start every trial from a clean environment
- Read the transcripts
- 42% → 95%: same model, only the eval was fixed

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

When not to use evals

An eval takes work up front and pays you back later.

Few uses, early days

A skill you run a handful of times may never pay back. Using it yourself and checking by hand gets you far.

No definition of good

Can't describe a good result yet? Write a few cases by hand first. That makes you decide.

Only the easy cases

Tasks picked because they're easy to make or grade don't measure what matters.

Too subjective to check

If nobody can check that the grader is right, use human review instead.

Never as the only signal: use evals together with monitoring, user feedback and real conversations

When not to use evals

Evals are not always worth it. An eval takes work up front and pays you back later, each time you change the skill or a new model comes out. If you only run a skill a few times, that payback may never come. Early on, using the skill yourself and checking the results by hand gets you a long way; Anthropic's post says the same.

Four cases where a full eval suite is not worth it yet. Few uses, early days: the work never pays back. No clear idea of a good result: you are not ready for a suite yet. Still write a few cases by hand, because that makes you decide what a good result is. Only the easy cases: sometimes tasks are picked because they are easy to make or easy to grade, not because they matter. Then the score measures the wrong thing. Too subjective to check: if nobody can check that the grader is right, use human review instead.

The main point: an eval is one signal. If it does not match how the skill is really used, it can make you think things work when they don't. Use it together with production monitoring, user feedback and reading real conversations.

Examples
• Few uses: a skill that moves the repo from Jest to Vitest runs once; check that diff by hand
• No definition of good: "better commit messages" needs 3 hand-written cases before the team knows what better means
• Only the easy cases: testing review-lessons only on "missing test" comments, because a regex catches them
• Too subjective: the tone of a release announcement; a person reads it
• Never the only signal: review-lessons passes its evals, but the team never acts on the lessons

When hillclimbing pays off

Raising a score only helps when it can still go up, and you can see why it moved.

- **Room to improve** · the score is well below ~95%. Above that, work on cost or speed instead.
- **Bigger than the noise** · the gain you want is bigger than the difference between runs.
- **One cheap change, one clear result** · the score moves because of that change.

Skills are a good fit. Whether a skill loads depends on its description, the one thing you edit.

No room to improve, or too much noise → stop

When hillclimbing pays off

Hillclimbing means: change one thing, run the eval again, and keep the change if the score goes up. It only makes sense when three things are true.

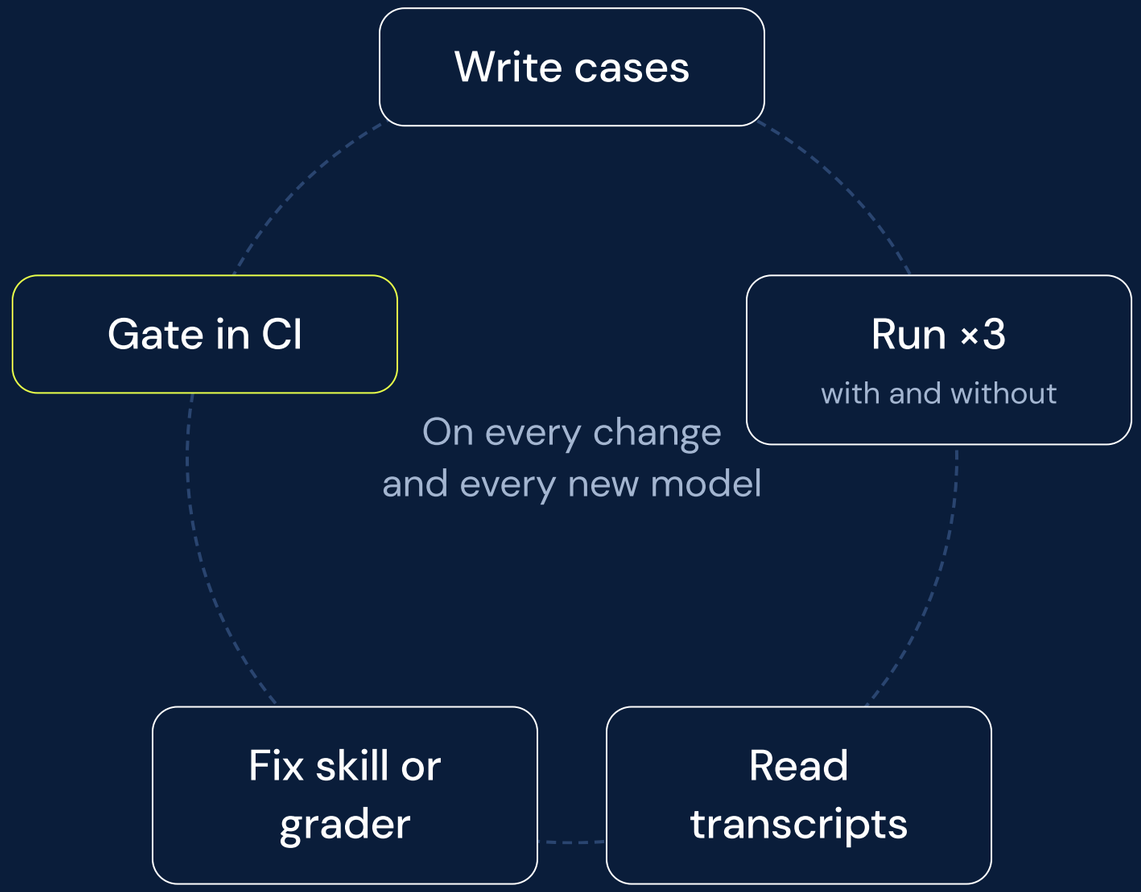
Room to improve: if the score is already around 95% or higher, more changes won't add much. Work on cost or speed instead. Bigger than the noise: agents give different results from run to run. If that difference is bigger than the improvement you want, you can't tell a real gain from luck. Run more trials, or stop. One cheap change, one clear result: it works best when a change is quick to make and you can see that the score moved because of that change.

Skills fit this well. Whether a skill loads depends on its description, so the trigger rate depends directly on the one thing you are changing. This is the loop from the 'Test both directions' slide: try a few versions of the description and score them on held-out queries.

Example
• The same description scores 70%, 75% and 80% on three runs with no change, so the noise is about 10 points. A new description at 78% could be luck; one at 95% is a real gain.

Key points
• Hillclimbing = change, run again, keep what raises the score
• Score already ~95% or more: work on cost or speed instead
• The gain must be bigger than the difference between runs

The eval loop



The eval loop

How it works day to day. Write cases, run them 3 times with and without the skill, read the transcripts, fix either the skill or the grader, then run the suite in CI. Repeat on every change to the skill and every time a new model comes out. The plugin eval docs list both: catching regressions when you change the plugin or when a new model is released.

Key points

- Write → run ×3 → read → fix → CI
- Rerun on every skill change
- Rerun on every new model

Sources

https://code.claude.com/docs/en/plugin-evals

Define done first

An eval is a spec you can run.

- Start small, from real failures
- Grade outcomes, read transcripts
- Measure what the skill adds

Define done first

Summary of part one. 'An eval is a spec you can run' is our line, not a quote; Anthropic's post says evals force teams to specify what success means. Start small with real failures, grade the real outcome, read transcripts, and measure what the skill actually adds compared with no skill.

Key points

- Decide what success means before building
- Start small, from real failures
- Grade outcomes and read transcripts
- Measure the difference the skill makes

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
https://code.claude.com/docs/en/plugin-evals

WUNDERDOG

02

Cloud agents

02 Cloud agents

Start of part two: agents that run without your laptop. Facts here come from the Claude Code docs on cloud sessions, environments and routines.

Key points

- Part 2: agents that keep working when you're away

Sources

https://code.claude.com/docs/en/claude-code-on-the-web
https://code.claude.com/docs/en/routines

From pair programmer to teammate



Autocomplete	Chat	Local agent	Cloud agent	Routines
Suggests the next line	Answers questions	Edits and runs code	Works while you're away	Starts on its own triggers
You type	You ask	You watch	You review	You set the goal

From pair programmer to teammate

How the way we use AI for coding has changed, step by step. Autocomplete suggests the next line. Chat answers questions. A local agent edits and runs code while you watch. A cloud agent works while you are away and you review the result. Routines start on their own from a schedule or an event. Each step gives the agent more freedom and moves you from typing to reviewing.

Key points

- Your role moves from typing to reviewing
- Cloud agents work without you present
- Routines start themselves from triggers

Sources

https://code.claude.com/docs/en/claude-code-on-the-web
https://code.claude.com/docs/en/routines

Where agents run

Local session

Your machine, your files, your tools

`claude`

Remote Control

Your machine, steered from phone or browser

`claude remote-control`

Self-hosted

Your infrastructure, cloud workflow

`self-hosted environment`

Cloud session

Managed VM, keeps going when the laptop closes

`claude --cloud`

Runs on your hardware

Runs in managed cloud

Where agents run

Four places Claude Code can run. Local session: on your machine with your files. Remote Control: still runs on your machine, but you control it from your phone or browser ('claude remote-control' starts server mode, 'claude --remote-control' an interactive session). Self-hosted environments: cloud sessions on infrastructure your organisation runs; public beta on Team and Enterprise, off by default. Cloud session: an Anthropic-managed VM that keeps going after you close the laptop, started with 'claude --cloud' ('--remote' still works as an old alias).

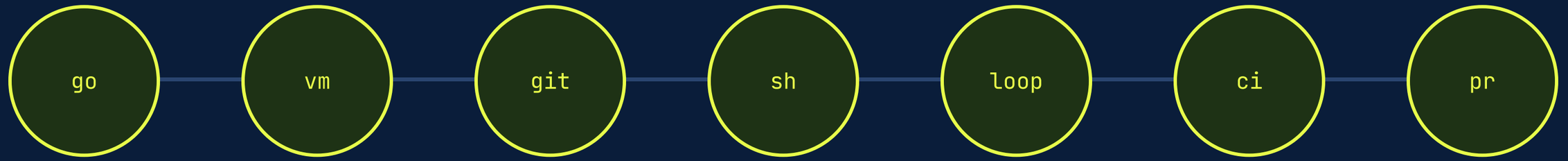
Key points

- Left side runs on your hardware, right side in managed cloud
- Remote Control: code never leaves your machine
- Self-hosted environments: public beta, Team and Enterprise
- claude --cloud starts a cloud session

Sources

https://code.claude.com/docs/en/remote-control
https://code.claude.com/docs/en/self-hosted-environments
https://code.claude.com/docs/en/claude-code-on-the-web

Anatomy of a cloud session



Trigger
Person, schedule, API, PR

Fresh VM
Isolated per session

Clone
Repo at branch

Setup
Script, cached

Agent loop
Edit, run, test

Checks
Tests and lint

Pull request
On a claude/ branch

SAME SHAPE — Claude Code · Codex Cloud · Copilot cloud agent · Cursor Cloud Agents

Anatomy of a cloud session

What happens in a cloud session. Something starts it: a person, a schedule, an API call or a GitHub event. It gets its own isolated VM. With --cloud it clones your GitHub repo at your current branch (routines start from the default branch). The setup script installs dependencies, the agent edits and runs code, checks run, and the result becomes a pull request. Routines push to branches starting with claude/. Other tools follow the same pattern: OpenAI Codex Cloud, GitHub Copilot cloud agent (earlier called coding agent) and Cursor Cloud Agents.

Key points

- One isolated VM per session
- Repo cloned from GitHub, not your local files
- Ends in a PR you review
- Same pattern across Codex, Copilot and Cursor

Sources

https://code.claude.com/docs/en/claude-code-on-the-web
https://code.claude.com/docs/en/routines
https://learn.chatgpt.com/docs/cloud
https://docs.github.com/en/copilot/how-tos/use-copilot-agents/coding-agent
https://cursor.com/docs/cloud-agent

The environment is the contract

NETWORK ACCESS

None | **Trusted** | Custom | Full

SETUP SCRIPT CACHED

```
npm ci && npx playwright install
```

VARIABLES

```
NODE_ENV=test
```

registry.npmjs.org	allowed
pypi.org	allowed
api.yourcompany.com	blocked
random-paste-site.io	blocked

Open only what the task needs

The environment is the contract

The environment decides what a cloud session is allowed to do. Network access has four levels: None, Trusted, Custom and Full. Trusted allows a default list: package registries like npm and PyPI, GitHub and cloud SDKs. Custom lets you add your own domains. The setup script runs the first time; if it finishes within about five minutes, the result is saved and reused, and it is rebuilt when the script or allowed hosts change or after about seven days. Environment variables are set per environment. In the demo, only the domains the task needs are green.

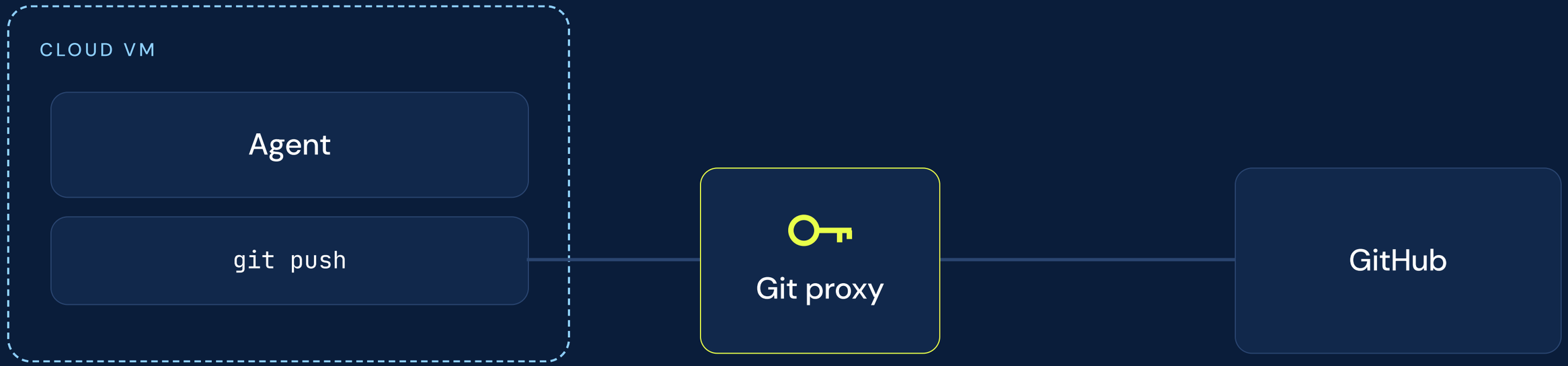
Key points

- None / Trusted / Custom / Full
- Trusted = package registries, GitHub, cloud SDKs
- Setup is cached if it finishes in about 5 minutes
- Open only what the task needs

Sources

https://code.claude.com/docs/en/cloud-environments

Credentials stay outside



- Tokens never enter the VM
- No tag pushes or branch deletes
- Branch protection rules still apply

Credentials stay outside

GitHub tokens never go inside the cloud VM. The agent runs git push as normal, and a GitHub proxy outside the VM adds the credentials. The proxy blocks branch deletes and pushes that are not branches, like tags. It does not limit which branches can be updated, so your GitHub branch protection rules are what protect main.

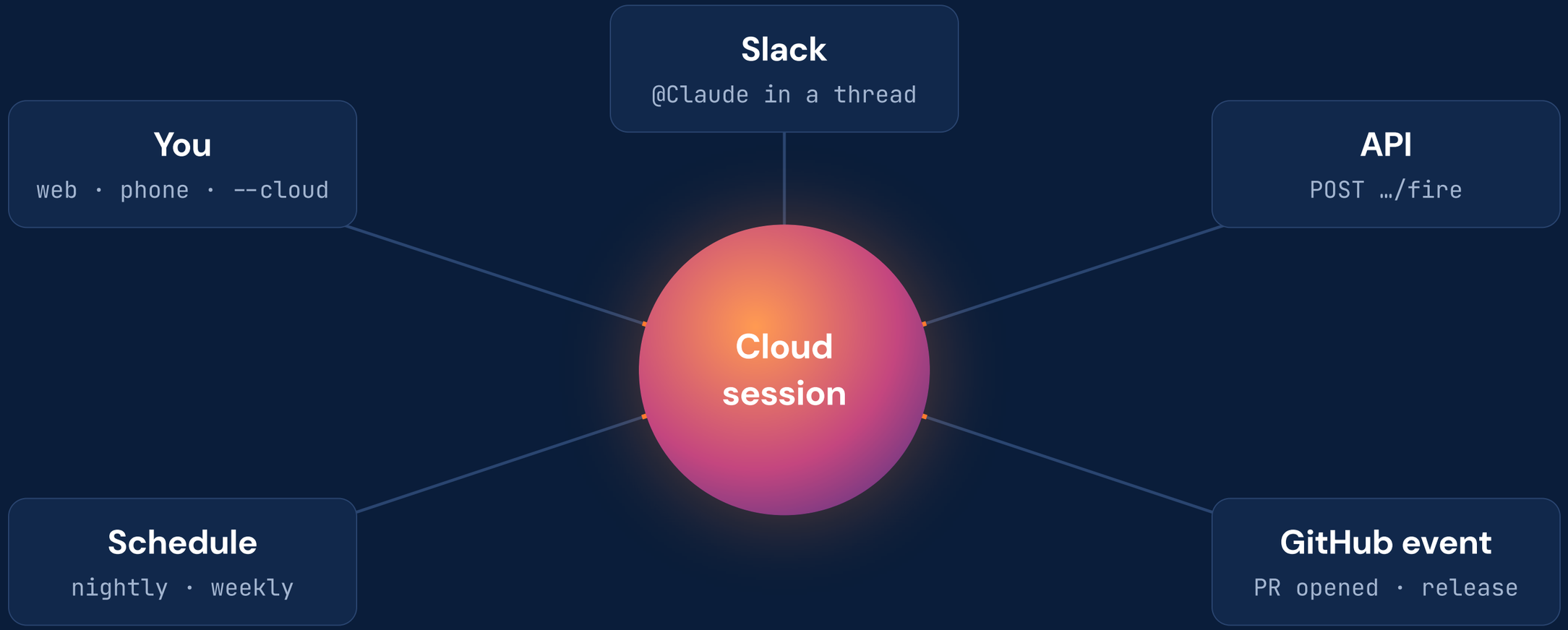
Key points

- Tokens stay outside the VM
- Proxy blocks tag pushes and branch deletes
- Branch protection on GitHub still matters

Sources

https://code.claude.com/docs/en/cloud-environments

What starts a cloud agent



What starts a cloud agent

Ways a cloud session starts. You: from claude.ai/code, the Claude mobile app, the desktop app or 'claude --cloud' in the terminal. Routines add three triggers: a schedule (minimum every hour), an API call (a POST to the routine's own /fire URL) and GitHub events (pull requests and releases, filtered by action like pull_request.opened). Slack: @Claude in a channel through Claude Tag, which runs on the same cloud setup; it is Team and Enterprise only and in public beta. Routines are in research preview.

Key points

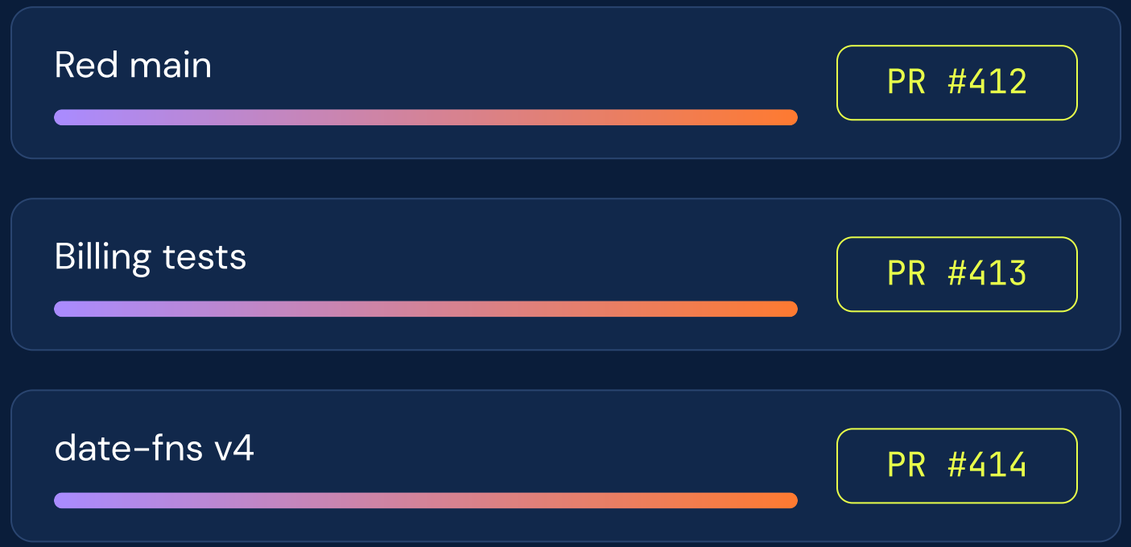
- You, schedule, API, GitHub, Slack
- Schedules run at most once an hour
- API trigger = POST to the routine's /fire endpoint
- Routines: research preview; Claude Tag: public beta

Sources

https://code.claude.com/docs/en/claude-code-on-the-web
https://code.claude.com/docs/en/routines
https://claude.com/docs/claude-tag/overview

Fan out

```
$ claude --cloud "Triage failed run 8812 on main"
$ claude --cloud "Add missing tests for billing"
$ claude --cloud "Upgrade date-fns to v4"
```



- One VM per task
- Parallel runs share your usage

Fan out

You can start several cloud sessions at once and each gets its own VM. Here three unrelated jobs run in parallel and each ends as its own PR. They all count against the same account rate limits, so more parallel sessions use your limits faster. There is no separate compute bill. The PR numbers are example data.

Key points

- One VM per task
- Runs in parallel, each ends in its own PR
- Shares your account's rate limits

Sources

https://code.claude.com/docs/en/claude-code-on-the-web

Plan locally, run in the cloud

LOCAL

Plan

```
claude --permission-mode plan
```

→

LOCAL

Commit the plan

```
docs/migration-plan.md
```

→

CLOUD

Execute

```
claude --cloud "Execute docs/migration-plan.md"
```

→

LOCAL

Pull it back

```
claude --teleport
```

- Talk where feedback is fast
- Run where waiting is free

Plan locally, run in the cloud

A practical workflow. Plan on your machine in plan mode ('claude --permission-mode plan'), where Claude cannot edit files until you approve the plan. Commit the plan as a file. Start a cloud session that executes it. When it's done, 'claude --teleport' brings the session into your terminal. Teleport is one way: work you do locally after that does not go back to the cloud session.

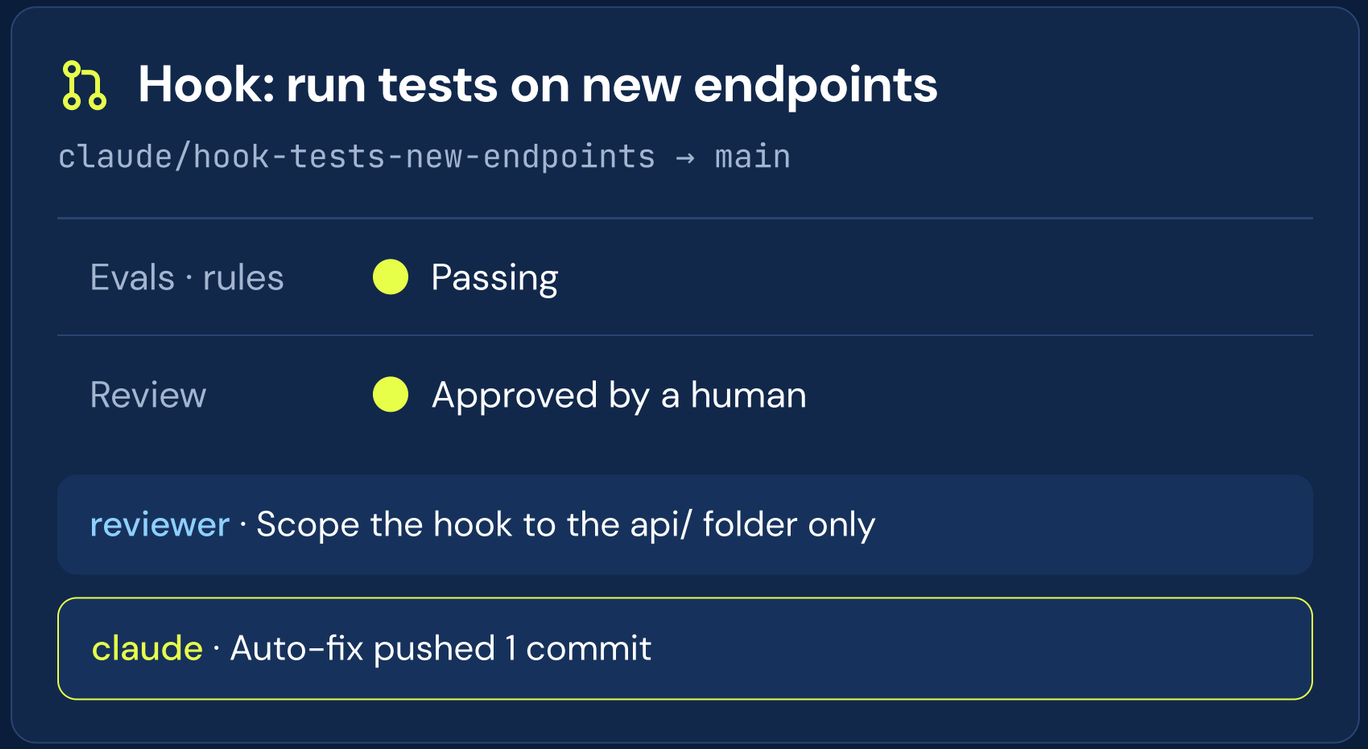
Key points

- Plan mode blocks edits until you approve
- Commit the plan so the cloud session can read it
- --teleport pulls the session back to your terminal
- Talk locally, wait in the cloud

Sources

https://code.claude.com/docs/en/permission-modes
https://code.claude.com/docs/en/claude-code-on-the-web

Contributors, not committers



- Agent branch, human merge
- Auto-fix answers CI and review comments
- Branch protection stays on

Contributors, not committers

Cloud agents open PRs like any contributor; they do not merge. Here a hook PR breaks one eval case, a human reviewer asks for a change, and Claude pushes a fix. That is the Auto-fix pull requests feature: Claude watches the PR's GitHub events and pushes clear fixes for CI failures and review comments, and asks first when a comment is unclear. It needs the Claude GitHub App, is turned on per PR and does not handle merge conflicts. A human still approves and merges. The PR content is example data.

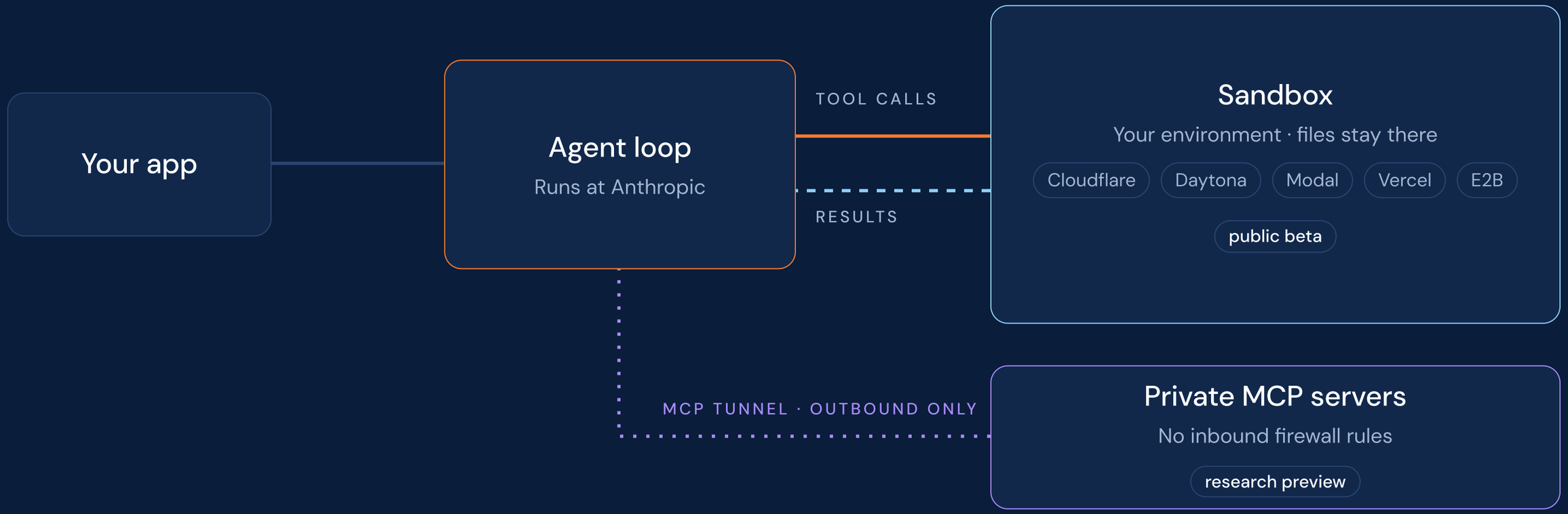
Key points

- Agent opens the PR, a human merges
- Auto-fix reacts to CI failures and review comments
- Needs the Claude GitHub App, per-PR toggle
- Keep branch protection on

Sources

https://code.claude.com/docs/en/claude-code-on-the-web

Build your own with Managed Agents



Build your own with Managed Agents

If you want this in your own product, Claude Managed Agents on the Claude Platform gives you the agent loop as a service. The loop runs at Anthropic; tool calls run in a sandbox. Self-hosted sandboxes (public beta) let those tools run in your own infrastructure so files stay with you; providers named include Cloudflare, Daytona, Modal, Vercel and E2B. MCP tunnels (research preview) connect private MCP servers with an outbound-only connection, so you do not open inbound firewall ports.

Key points

- Agent loop at Anthropic, tools in a sandbox
- Self-hosted sandboxes: public beta
- MCP tunnels: outbound only, research preview

Sources

https://platform.claude.com/docs/en/managed-agents/self-hosted-sandboxes
https://claude.com/blog/claude-managed-agents-updates
https://platform.claude.com/docs/en/agents-and-tools/mcp-tunnels/overview

03

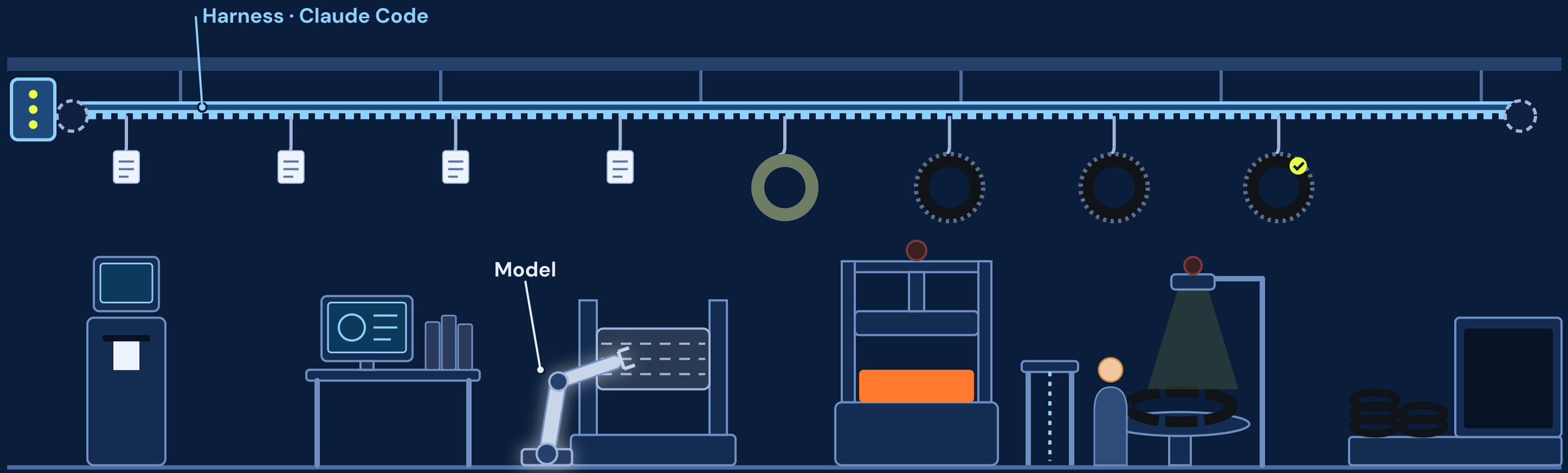
One plant, two lines

03 One plant, two lines

Part three puts everything together on one picture: a tire plant that stands for the agentic software process. Then the two workshop flows run on it: review lessons and CI health. The plant is just a drawing to keep the steps in order; the notes on each slide say exactly what each part means.

Key points
• One picture for the whole process
• Two flows: review lessons and CI health

Meet the tire plant



TIRE PLANT

Robot arm does the building; swap it and the line still runs

AGENTIC SDLC

Model does the work; the harness stays the same

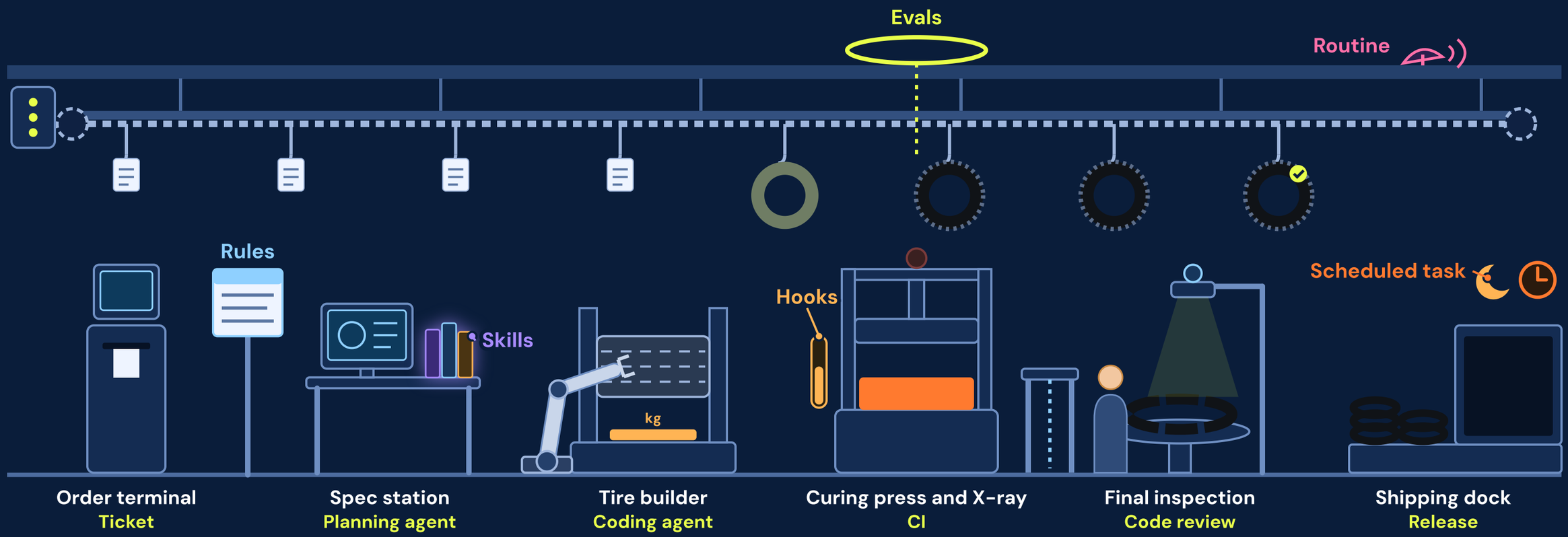
Meet the tire plant

The six stations are the six steps of an agentic development flow. Order terminal = ticket. Spec station = planning agent. Tire builder = coding agent. Curing press and X-ray = CI. Final inspection = code review. Shipping dock = release. The overhead chain is the harness, Claude Code, which moves every task through every step and gives it context, tools and permissions. The robot arm is the model. You can switch the model and the harness stays the same.

Key points

- Ticket → planning → coding → CI → review → release
- Chain = harness (Claude Code)
- Robot arm = model, it can be swapped

What runs the plant



TIRE PLANT

Control centre runs the plant while you are off site

AGENTIC SDLC

Routine in the cloud, on a schedule, an API call or a GitHub event

What runs the plant

What each extra on the plant stands for. Standards board = rules in CLAUDE.md, loaded at the start of every session. Recipe binders = skills; only their descriptions load up front and the full skill loads when needed. Scale and gauge = hooks, commands that run automatically at fixed points like before or after a tool call. Test track = evals. Night shift = a scheduled task on your own machine (desktop scheduled tasks can run as often as every minute). Control centre = a routine in the cloud, started by a schedule, an API call or a GitHub event.

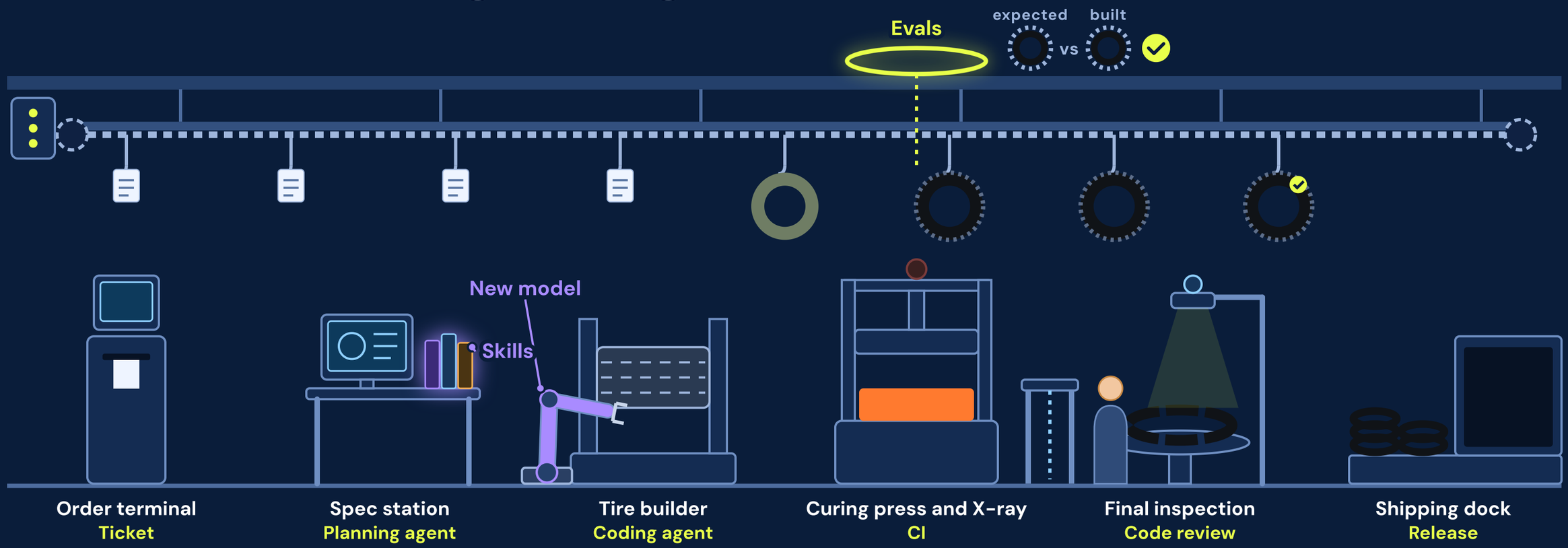
Key points

- CLAUDE.md is loaded every session
- Skills load on demand
- Hooks run every time at set points, not when the model decides
- Scheduled task = your machine; routine = cloud, minimum hourly

Sources

https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/hooks
https://code.claude.com/docs/en/scheduled-tasks
https://code.claude.com/docs/en/routines

When something changes



TIRE PLANT

Recipe updated for the new robot: right tread at the new speed

AGENTIC SDLC

Skill updated, evals green keep the speed-up, lose the drift

When something changes

Why evals matter when something changes. A new model (the new robot arm) or a changed spec makes the line faster, but the output changes too: the tread pattern is now different. Code review still lets it through because each single diff looks fine. The evals compare the result with the expected outcome and hold the release. After the skill is updated and the evals pass again, you keep the faster model without the wrong output. The plugin eval docs name this use: catching regressions when a new model is released.

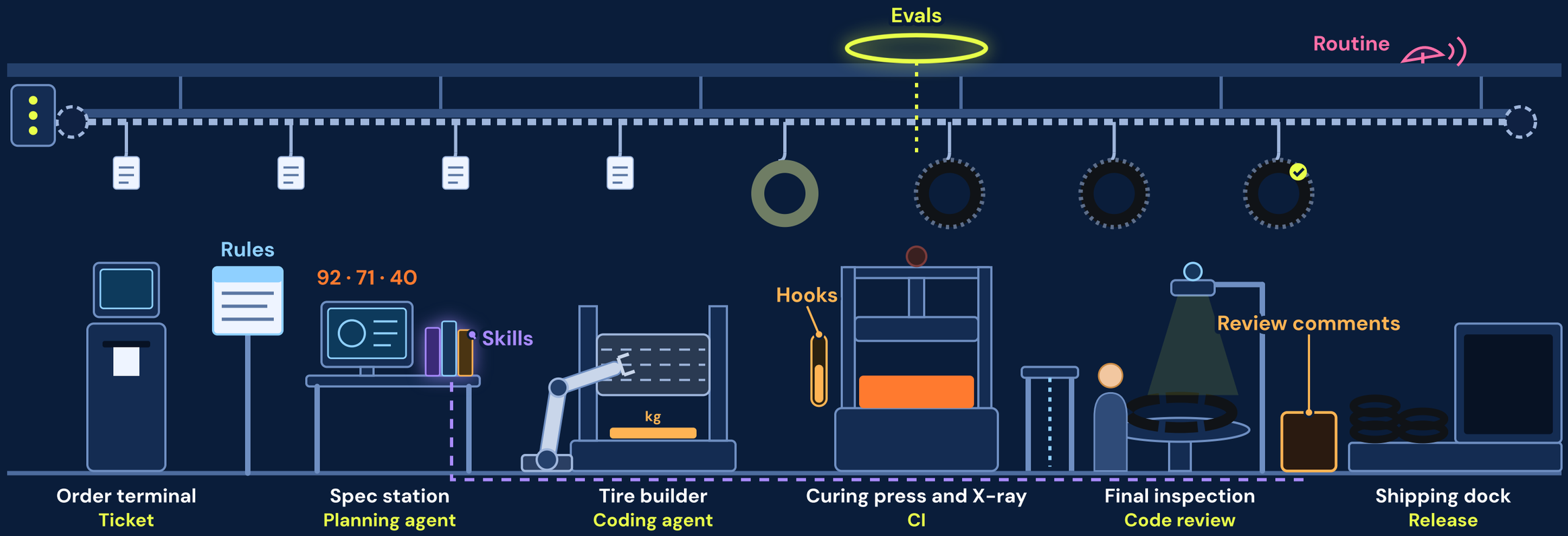
Key points

- A new model or spec can change the output quietly
- Review checks one diff, evals check the behaviour
- Evals hold the release until results match
- Fix the skill, keep the upgrade

Sources

https://code.claude.com/docs/en/plugin-evals

Line 1: Review lessons



TIRE PLANT

Weekly from the control centre nobody has to remember

AGENTIC SDLC

Routine weekly: posts the report to the team, opens no PR

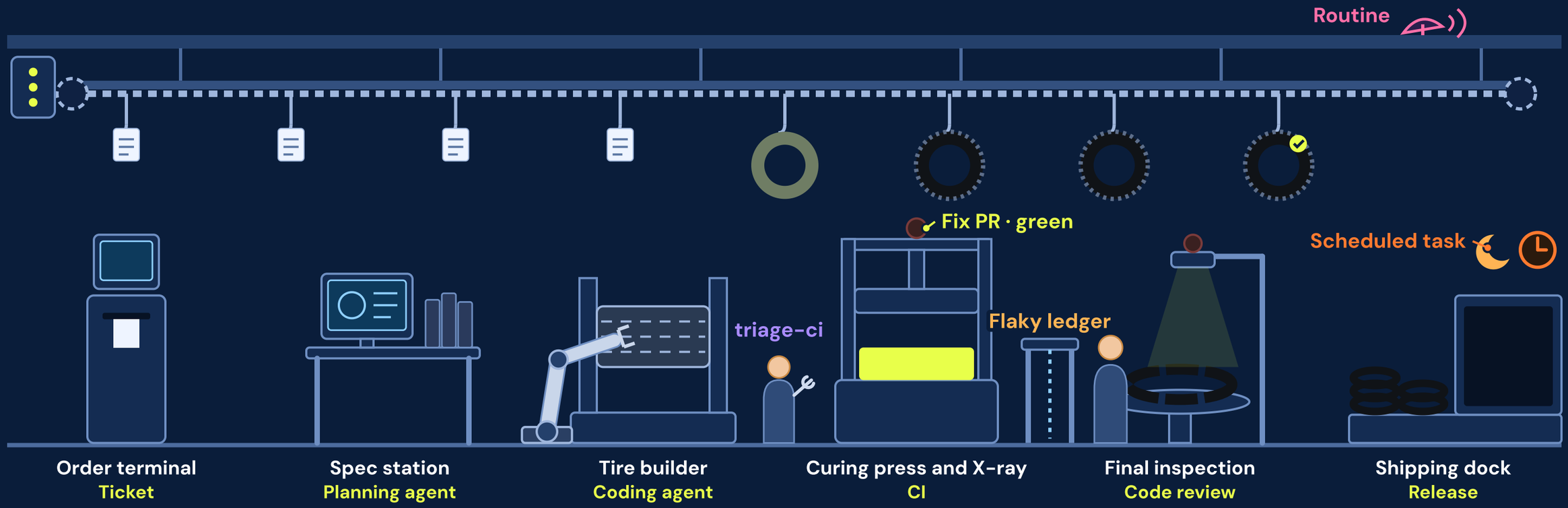
Line 1: Review lessons

Our main workshop flow. A skill reads the 'changes requested' comments from last week's merged PRs, groups them into lessons by problem type and counts them, then maps each problem type to an existing rule, skill or hook (or notes that none exists). It scores each suggested improvement from 100 to 0 by how many review comments it would prevent, highest first. A change only lands if the evals improve. Later a weekly routine runs it and posts the report to the team; it opens no PR. This is our design for the workshop, not an existing product; the counts on screen are example data.

Key points

- Input: review comments from merged PRs
- Output: lessons mapped to rules, skills, hooks
- Scored 100–0 by impact
- Eval gate stops bad suggestions
- Weekly routine posts the report to the team

Line 2: CI health



TIRE PLANT

Repair order press back in spec

AGENTIC SDLC

Draft fix PR a human reviews it; main is green again

Line 2: CI health

The second workshop flow. When main goes red, a triage-ci skill reads the logs and sorts the failure into flaky, broken or infrastructure. Flaky tests go into a ledger that tracks repeat offenders across runs. A scheduled task checks main every morning; a routine with an API trigger can start the same work in the cloud when CI fails. The result is a draft fix PR that a human reviews. Claude Code's auto-fix already handles CI on a single PR; this flow watches main over time. triage-ci is our workshop skill, not a built-in one.

Key points

- Diagnose before rerunning
- Ledger tracks flaky tests over time
- Morning scheduled task, cloud routine on CI failure
- Ends in a draft PR for a human

Sources

https://code.claude.com/docs/en/claude-code-on-the-web
https://code.claude.com/docs/en/routines

From skill to night shift to routine

	Skill On demand	Scheduled task Your machine	Routine Cloud
Review lessons	`/review-lessons` Before a retro	Friday afternoon: last week's comments	Weekly schedule: posts the report to the team
CI health	`/triage-ci <run>` When a build fails	Every morning: scan main, update the ledger	CI failure calls the API trigger; nightly ledger

Evals gate every step up

From skill to night shift to routine

Both flows grow the same way. Step 1: a skill you run by hand (/review-lessons before a retro, /triage-ci when a build fails). Step 2: a scheduled task on your machine that runs it at set times. Step 3: a routine in the cloud that runs without your machine, on a schedule (at most hourly) or from an API call. Evals must pass before each step up, because each step means less human checking.

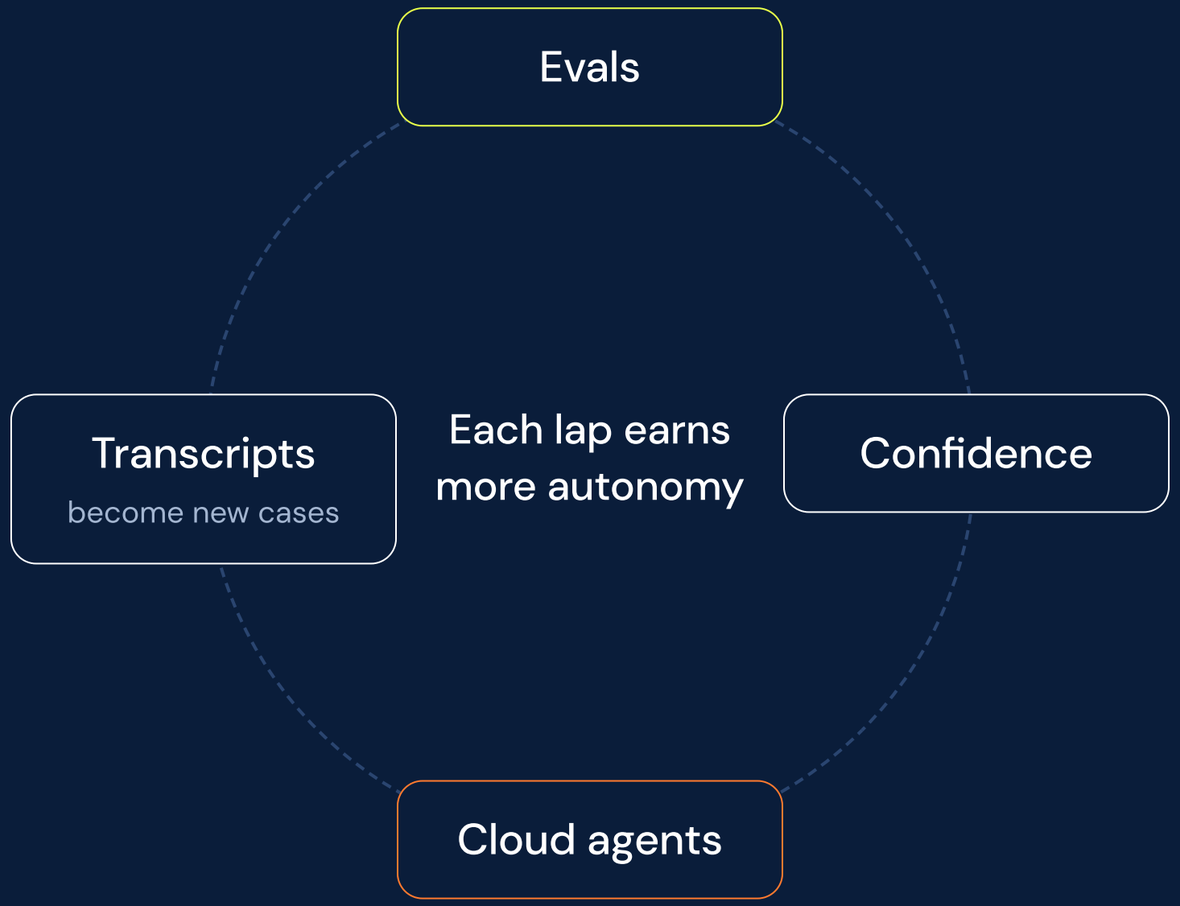
Key points

- Skill → scheduled task → routine
- Each step needs less of you
- Evals must pass before moving up

Sources

https://code.claude.com/docs/en/scheduled-tasks
https://code.claude.com/docs/en/routines

Trust before autonomy



Trust before autonomy

The loop behind the whole talk. Evals give you confidence. Confidence lets you hand more work to cloud agents. Their transcripts show new failures, and those become new eval cases. Each time around, the agent earns a bit more freedom. Anthropic's post makes the same point: read transcripts and turn real failures into tasks.

Key points

- Evals → confidence → cloud agents → transcripts → new cases
- Freedom grows step by step

Sources

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

More lines for the same plant

Library updates

One upgrade per PR, changelog read, tests green

`schedule · weekly`

SDLC observability

Alert in, investigation done, draft fix PR out

`API trigger · on alert`

Migrations at scale

Playbook as a skill, evals first, then fan out

`fan-out · per module`

Same machines. Same control system. New recipe.

More lines for the same plant

The same building blocks work for other jobs. Library updates: a weekly schedule, one upgrade per PR, changelog read, tests green. SDLC observability: an alert calls a routine's API trigger, Claude investigates and opens a draft fix PR. Migrations at scale: write the playbook as a skill, write evals first, then run many cloud sessions, one per module. These are ideas for after the workshop, not finished tools.

Key points

- Same skills, evals, hooks, schedules, routines
- Different job each time
- Ideas to take home

Sources

https://code.claude.com/docs/en/routines

https://code.claude.com/docs/en/claude-code-on-the-web

Thank you

WUN
DER
DOG

Thank you

Thank the audience, take questions, then point them to the workshop setup.

Key points
- Questions
- Next: workshop setup